Auto-discovering meson tests

Rohit Goswami · 2023-03-22 · meson · cpp · build · tests

One of the things I missed when I migrated from `cmake` to `meson` was the ease at which `cmake` discovers tests.

```
# Tests
option(PACKAGE_TESTS "Build the tests" OFF)
if(PACKAGE_TESTS)
  find_package(GTest REQUIRED)
  enable_testing()
  include(GoogleTest)
  add_subdirectory(gtests)
endif()
```

Thankfully, `meson` can kind of emulate this behavior, even in its restricted syntax. The key concept is **arrays** and their iterators.

```
test_array = [#
  # ['Pretty name', 'binary_name', 'BlahTest.cpp']
  ['String parser helpers', 'strparse_run', 'StringHelpersTest.cpp'],
  ['Improved Dimer', 'impldim_run', 'ImpDimerTest.cpp'],
]
foreach test : test_array
  test(test.get(0),
       executable(test.get(1),
          sources : ['gtests/'+test.get(2)],
          dependencies : [ prog_deps, gtest_dep, gmock_dep ],
          link_with : proglib,
          cpp_args : prog_extra_args
                 ),
      )
endforeach
```

Now this is pretty neat already, but we can go a step further and augment this with a working directory concept.

```
test_array = [#
  # ['Pretty name', 'binary_name', 'BlahTest.cpp', 'working_dir']
  ['Improved Dimer', 'impldim_run', 'ImpDimerTest.cpp', '/gtests/data/'],
  ['Matter converter', 'matter_run', 'MatterTest.cpp', '/gtests/data/systems/sulfolene'],
  ['String parser helpers', 'strparse_run', 'StringHelpersTest.cpp', ''],
]
foreach test : test_array
  test(test.get(0),
       executable(test.get(1),
          sources : ['gtests/'+test.get(2)],
          dependencies : [ prog_deps, gtest_dep, gmock_dep ],
          link_with : proglib,
          cpp_args : prog_extra_args
                 ),
        workdir : meson.source_root() + test.get(3)
      )
endforeach
```

This is rather satisfying. Combined with some `wraps`, it is also pretty portable.

Addenum: Tests with arguments

Recently, some `openblas` work made me rethink how `meson` handles tests and arguments. It turns out something relatively simple in `make`:

```
xblat2s: sblat2.o $(BLASLIB)
        $(FC) $(FFLAGS) $(LDFLAGS) -o $@ $^
run: all
        ./xblat2s < sblat2.in
```

Is rather complicated to emulate directly within `meson`. A wrapper is required, first in `C`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <executable> <input_file>\n", argv[0]);
        return EXIT_FAILURE;
    }

    char command[1024];
    snprintf(command, sizeof(command), "%s < %s", argv[1], argv[2]);

    int result = system(command);
    if (result != 0) {
        fprintf(stderr, "Error: Command '%s' failed with return code %d.\n", command, result);
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Which can then be used within `meson`:

```
_blas_input_test_array = [#
  # ['Pretty name', 'binary_name', 'BlahTest.cpp', 'inputfile.in']
  ['Test REAL Level 2 BLAS', 'xblat2s', 'sblat2.f', 'sblat2.in'],
]
test_runner = executable('run_test',
                         sources: ['run_test.c'],
                         install: false)
# NOTE: For the tests to pass the executables need to be compiled first
foreach _test : _blas_input_test_array
  executable(_test.get(1),
             sources : _test.get(2),
             link_with : netlib_blas,
            )
  test_exe = meson.current_build_dir() / _test.get(1)
  input_file = meson.source_root() + '/lapack-netlib/BLAS/TESTING/' + _test.get(3)
  test(_test.get(0), fortran_test_runner,
       args: [test_exe, input_file],
       workdir: meson.current_build_dir())
endforeach
```

However, the main pain point with this is that the executables are not exactly test dependencies so this fails if the entire project hasn't been compiled before running the tests. Still a pretty minor caveat.